Web Technology (BCS-502)

Unit 1: Web Foundations, HTML & XML

Q1. Short Definitions (2 Marks Each)

(a) Define HTTP and mention its three basic features.

- **HTTP (HyperText Transfer Protocol)** is an **application-layer, request-response protocol** used to transfer hypertext (HTML, images, data) between a web **client (browser)** and a **web server**. It runs over TCP, on **port 80** by default (HTTPS uses port 443).
- **Three basic features:**
    1. **Connectionless:** the client sends a request, the server responds, and the connection is closed. A new connection is used for the next request (in HTTP/1.0).
    2. **Stateless:** the server keeps **no memory** of previous requests. Each request is independent, so cookies and sessions are used to keep state.
    3. **Media independent:** any type of data (text, image, video, JSON) can be sent, as long as both sides understand the **MIME type** given in the `Content-Type` header.

(b) Differentiate between HTML and XML.

Basis	HTML	XML
Full form	HyperText Markup Language	eXtensible Markup Language
Purpose	**Displays** data (presentation)	**Stores and transports** data
Tags	**Predefined** (`<p>`, `<h1>`)	**User-defined** (`<book>`, `<author>`)
Case sensitivity	Not case sensitive	**Case sensitive**
Closing tags	Some optional (` `, `<li>`)	**Mandatory** for every element
Error handling	Lenient (browser ignores errors)	**Strict** (must be well-formed)
Whitespace	Not preserved	Preserved

(c) Difference between Client-Server and Peer-to-Peer paradigms.

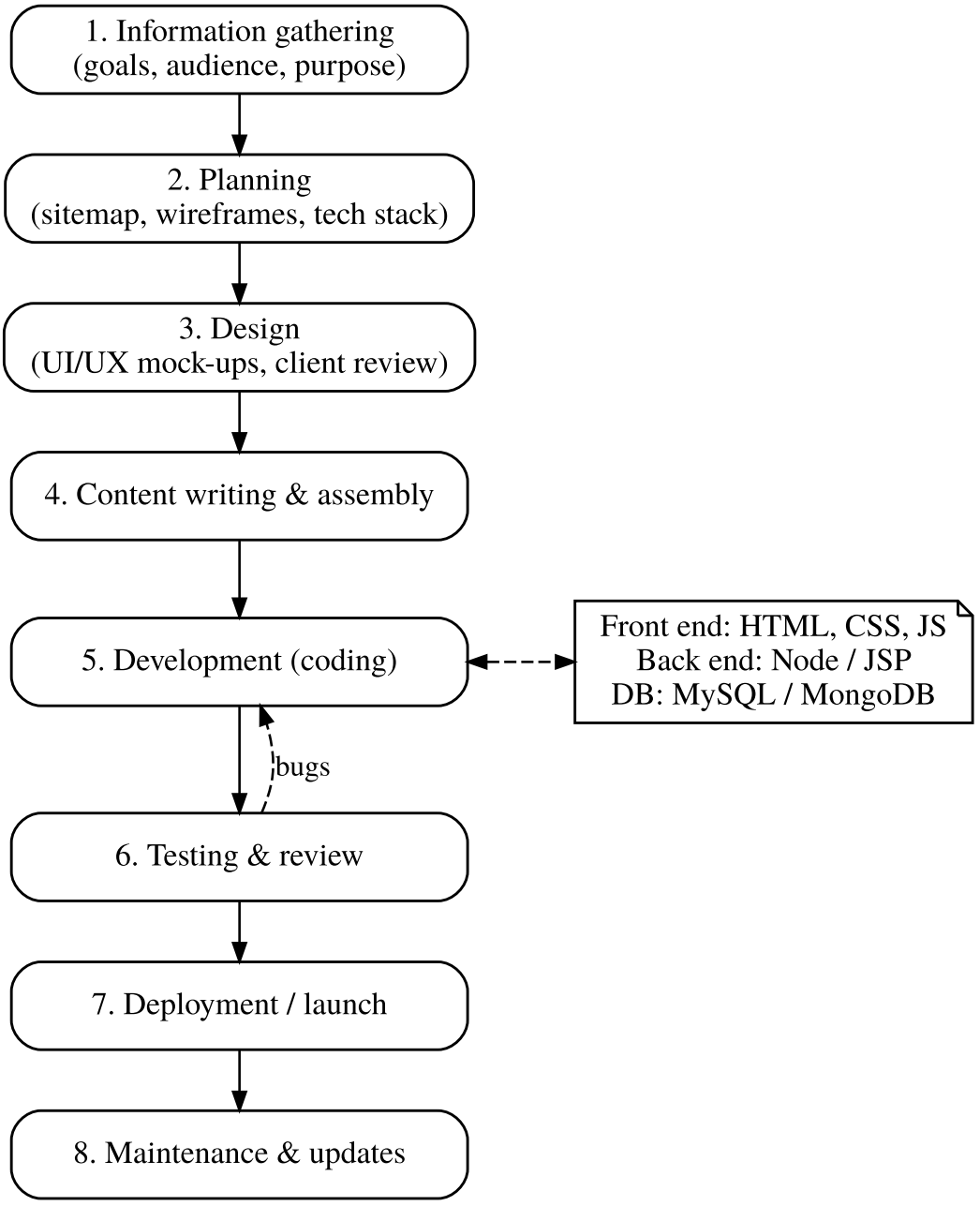
Client-Server	Peer-to-Peer (P2P)
A **dedicated server** provides services and clients request them	**Every node (peer)** acts as both client and server
Centralised control and data	**Decentralised**, with data distributed among peers
The server is a **single point of failure**	**No single point of failure**
Easy to secure and back up	Hard to secure and manage
Costly (needs server hardware)	Cheap (uses peers' resources)
Example: Web (browser to Apache), email	Example: BitTorrent, blockchain, Skype (early)

Q2. Explain web development strategies and the web development process using a block diagram. (7 Marks)

1. Introduction / Definition

Web development is the process of building, testing and maintaining websites and web applications. A **web development strategy** is the planned approach (goals, audience, technology and methodology) followed to deliver a site that is **usable, scalable and secure**. The **web development process** is the step-by-step life cycle from planning to maintenance.

2. Block Diagram - Web Development Life Cycle



Web development life cycle

3. Core Explanation / Working

(a) Web Development Strategies

- **Define goals and audience:** decide what the site must achieve (sales, information, service) and for whom.
- **Choose an architecture:** a static site, a dynamic site (server-side rendering), or a **single-page application (SPA)**.
- **Mobile-first / responsive design:** design for small screens first, then scale up with CSS media queries.
- **Choose a methodology:**
  - **Waterfall:** sequential phases, suited to fixed requirements.
  - **Agile / Scrum:** short iterative sprints with frequent client feedback.
  - **Rapid prototyping:** build a quick model, take feedback, refine.
- **Technology stack selection:** for example **MERN** (MongoDB, Express, React, Node) or **LAMP** (Linux, Apache, MySQL, PHP).
- **SEO, accessibility and security** are planned from the start, not added at the end.

(b) Web Development Process (as in the diagram)

1. **Information gathering:** understand the purpose, the audience and the content needed.
2. **Planning:** build a **sitemap** (page hierarchy) and **wireframes** (page skeletons). Choose tools.
3. **Design:** create the visual layout and UI mock-ups, and get approval.
4. **Content creation:** write text, collect media, optimise for SEO.
5. **Development:** code the front end (HTML, CSS, JS) and the back end (server logic, database, APIs).
6. **Testing:** check links, forms, browser compatibility, responsiveness, load and security.
7. **Deployment:** upload to a web server or cloud, and configure the domain and SSL.
8. **Maintenance:** fix bugs, update content and monitor performance.

4. Key Characteristics - Waterfall vs. Agile Strategy

Basis	Waterfall	Agile
Flow	Linear, sequential	Iterative, incremental
Client feedback	At the end	After every sprint
Change handling	Difficult and costly	Easy
Delivery	One final release	Frequent working releases
Best for	Small, fixed-scope sites	Large, evolving web apps

5. Advantages & Disadvantages (of following a structured process)

Advantages

- **Clear planning** reduces cost overruns and missed deadlines.
- Early testing gives a **higher-quality**, bug-free product.
- A responsive, SEO-friendly design improves **user reach**.

Disadvantages

- A structured process takes **more time** at the start (planning, documentation).
- With Waterfall, **late requirement changes** are expensive.

6. Conclusion / Application

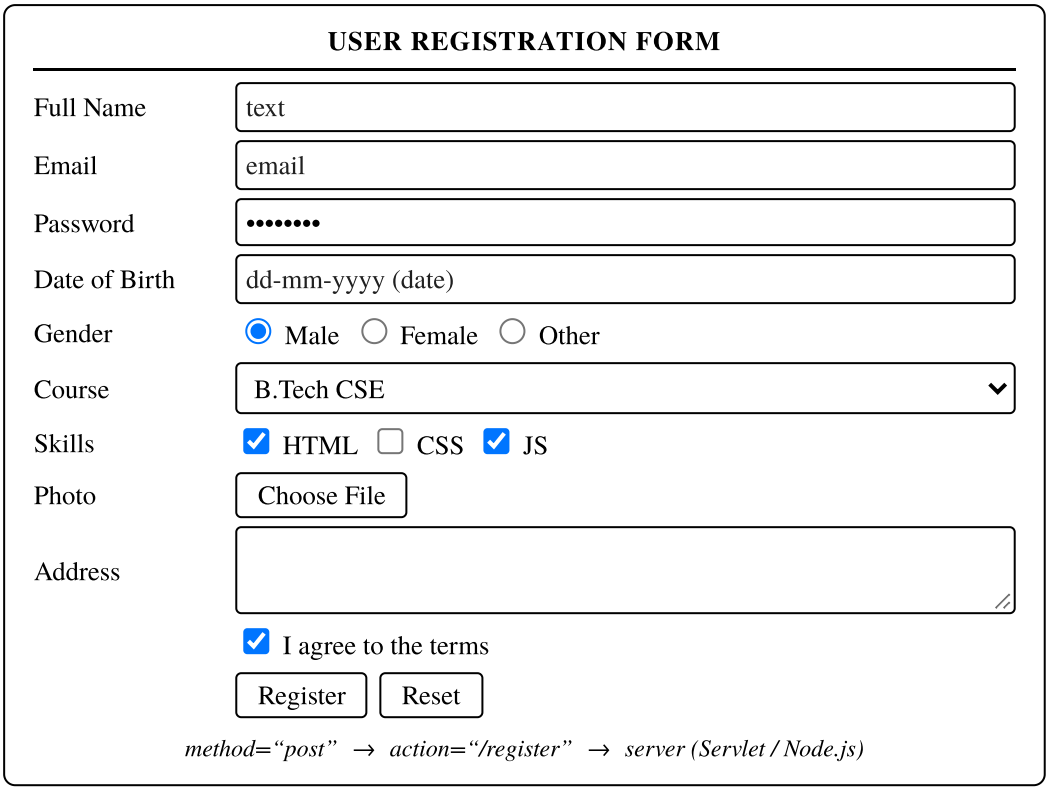
This process is followed by every web agency and product team, for example when building an **e-commerce site like Flipkart** or a **college ERP portal**.

Q3. Write the HTML code to create a comprehensive user registration form (using various input types and action attributes). (7 Marks)

1. Introduction / Definition

An **HTML form** collects user input and sends it to a server for processing. It is created with the `<form>` element. The **`action`** attribute gives the **URL** that will process the data, and the **`method`** attribute (`GET` or `POST`) decides **how** the data is sent.

2. Block Diagram - Form Layout and Data Flow



Layout of the registration form and where its data goes

3. Core Explanation / Working - HTML Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>User Registration</title>
</head>
<body>
  <h2>User Registration Form</h2>

  <form action="/register" method="post" enctype="multipart/form-data">
    <fieldset>
      <legend>Personal Details</legend>

      <label for="name">Full Name:</label>
      <input type="text" id="name" name="fullname"
             placeholder="Enter full name" maxlength="50" required><br><br>

      <label for="email">Email:</label>
      <input type="email" id="email" name="email" required><br><br>

      <label for="pwd">Password:</label>
      <input type="password" id="pwd" name="password"
             minlength="8" required><br><br>

      <label for="dob">Date of Birth:</label>
      <input type="date" id="dob" name="dob"><br><br>

      <label for="mob">Mobile:</label>
      <input type="tel" id="mob" name="mobile"
             pattern="[0-9]{10}" title="10-digit number"><br><br>

      <label for="age">Age:</label>
      <input type="number" id="age" name="age" min="17" max="60"><br><br>

      Gender:
      <input type="radio" id="m" name="gender" value="male" checked>
      <label for="m">Male</label>
      <input type="radio" id="f" name="gender" value="female">
      <label for="f">Female</label>
      <input type="radio" id="o" name="gender" value="other">
      <label for="o">Other</label><br><br>
    </fieldset>

    <fieldset>
      <legend>Academic Details</legend>

      <label for="course">Course:</label>
      <select id="course" name="course">
        <option value="cse">B.Tech CSE</option>
        <option value="it">B.Tech IT</option>
        <option value="ece">B.Tech ECE</option>
      </select><br><br>

      Skills:
      <input type="checkbox" name="skills" value="html"> HTML
      <input type="checkbox" name="skills" value="css"> CSS
      <input type="checkbox" name="skills" value="js"> JavaScript<br><br>

      <label for="photo">Upload Photo:</label>
      <input type="file" id="photo" name="photo" accept="image/*"><br><br>

      <label for="site">Portfolio URL:</label>
      <input type="url" id="site" name="website"><br><br>

      <label for="addr">Address:</label><br>
      <textarea id="addr" name="address" rows="4" cols="40"></textarea><br><br>
```

```
            <input type="hidden" name="source" value="college_portal">
            <input type="checkbox" id="agree" name="agree" required>
            <label for="agree">I agree to the terms and conditions</label><br><br>
        </fieldset>

        <input type="submit" value="Register">
        <input type="reset" value="Reset">
    </form>
</body>
</html>
```

Explanation of the key attributes

- `action="/register"`: the server URL that receives the form data.
- `method="post"`: sends the data in the **request body**, which is secure for passwords. `get` would append it to the URL.
- `enctype="multipart/form-data"`: **required** for file uploads.
- `name`: the key under which each value is sent (for example `email=abc@x.com`).
- `required`, `pattern`, `min`, `max`, `maxlength`: built-in **HTML5 validation**.
- `<fieldset>` and `<legend>` group related fields. `<label for>` links a label to its input.

4. Key Characteristics - Input Types Used

Input Type	Purpose	Built-in Validation
`text`	Single-line text	maxlength, pattern
`email`	Email address	Checks the @ format
`password`	Hidden characters	minlength
`date`	Date picker	Valid date
`tel`	Phone number	pattern
`number`	Numeric input	min, max
`radio`	Choose one option	Same `name` groups them
`checkbox`	Choose many options	-
`file`	Upload file	accept
`url`	Web address	Checks URL format
`hidden`	Value sent without display	-
`submit` / `reset`	Send / clear the form	-

5. Advantages & Disadvantages

Advantages

- HTML5 input types give **built-in validation** and suitable keyboards on mobile.
- Forms are a **standard, browser-independent** way to collect user data.
- `POST` with `multipart/form-data` supports **secure and binary (file)** submissions.

Disadvantages

- Client-side validation can be **bypassed**, so server-side validation is still required.

- Older browsers may **not support** newer input types like `date`.

6. Conclusion / Application

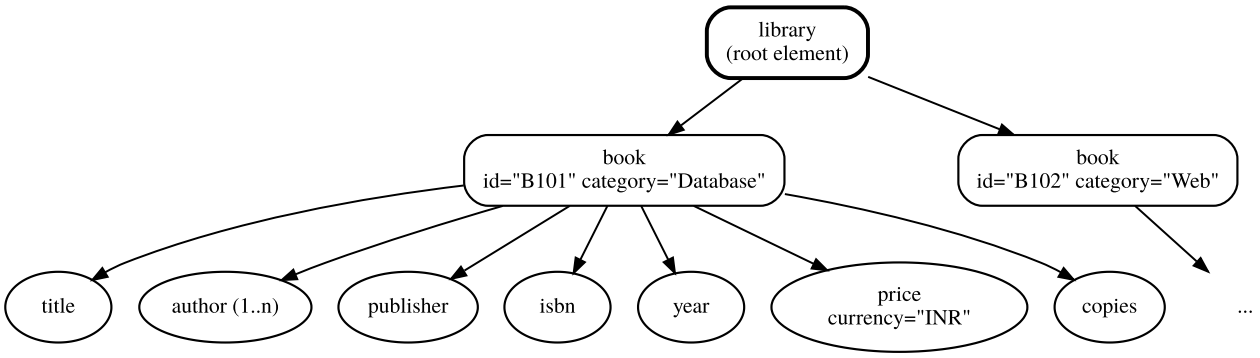
Registration forms like this are used on every **sign-up page**, for example college admission portals, Gmail sign-up and e-commerce account creation.

Q4. Write an XML file to store library book information and design a valid XML DTD or XML Schema for it. (7 Marks)

1. Introduction / Definition

XML (eXtensible Markup Language) stores and transports data in a **self-describing, hierarchical** format with user-defined tags. A **DTD (Document Type Definition)** or an **XML Schema (XSD)** defines the **legal structure** of an XML document: its elements, attributes, order and data types. A document that follows its DTD or XSD is called **valid**.

2. Block Diagram - XML Tree Structure



XML tree of library.xml, validated against a DTD or XSD

3. Core Explanation / Working

(a) XML File: `library.xml` (with an internal DTD)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE library [
  <!ELEMENT library (book+)>
  <!ELEMENT book (title, author+, publisher, isbn, year, price, copies)>
  <!ELEMENT title     (#PCDATA)>
  <!ELEMENT author    (#PCDATA)>
  <!ELEMENT publisher (#PCDATA)>
  <!ELEMENT isbn      (#PCDATA)>
  <!ELEMENT year      (#PCDATA)>
  <!ELEMENT price     (#PCDATA)>
  <!ELEMENT copies    (#PCDATA)>
  <!ATTLIST book id       ID    #REQUIRED
                 category CDATA #IMPLIED>
  <!ATTLIST price currency CDATA "INR">
]>
<library>
  <book id="B101" category="Database">
    <title>Database System Concepts</title>
    <author>Abraham Silberschatz</author>
    <author>Henry F. Korth</author>
    <publisher>McGraw Hill</publisher>
    <isbn>9780073523323</isbn>
    <year>2019</year>
    <price currency="INR">850</price>
    <copies>12</copies>
  </book>
  <book id="B102" category="Web">
    <title>Web Technologies</title>
    <author>Uttam K. Roy</author>
    <publisher>Oxford</publisher>
    <isbn>9780198066224</isbn>
    <year>2018</year>
    <price currency="INR">560</price>
    <copies>8</copies>
  </book>
</library>
```

DTD symbols used

- `book+`: one or more book elements. (* means zero or more, ? means zero or one.)
- `#PCDATA`: parsed character data (text).
- `ID #REQUIRED`: a unique, mandatory attribute. `#IMPLIED` means optional. `"INR"` is the default value.

(b) Equivalent XML Schema: `library.xsd`

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="library">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="book" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="title"     type="xs:string"/>
              <xs:element name="author"    type="xs:string" maxOccurs="unbounded"/>
              <xs:element name="publisher" type="xs:string"/>
              <xs:element name="isbn"      type="xs:string"/>
              <xs:element name="year"      type="xs:gYear"/>
              <xs:element name="price">
                <xs:complexType>
                  <xs:simpleContent>
                    <xs:extension base="xs:decimal">
                      <xs:attribute name="currency" type="xs:string" default="INR"/>
                    </xs:extension>
                  </xs:simpleContent>
                </xs:complexType>
              </xs:element>
              <xs:element name="copies" type="xs:nonNegativeInteger"/>
            </xs:sequence>
            <xs:attribute name="id"       type="xs:ID"     use="required"/>
            <xs:attribute name="category" type="xs:string" use="optional"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Linking the XSD to the XML file (instead of the internal DTD):

```
<library xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="library.xsd">
```

4. Key Characteristics - DTD vs. XML Schema

Basis	DTD	XML Schema (XSD)
Syntax	Non-XML (special syntax)	Written **in XML** itself
Data types	Only text (#PCDATA, CDATA)	**44+ built-in types** (int, date, decimal)
Namespaces	Not supported	Supported
Occurrence control	?, *, + only	`minOccurs`, `maxOccurs` (exact numbers)
Extensibility	Not extensible	Extensible (inheritance of types)
Complexity	Simple	More verbose and complex

5. Advantages & Disadvantages

Advantages (of XML with DTD/XSD)

- **Platform independent**, so data can be exchanged between different systems.
- **Validation** catches structural errors before processing.
- **Self-describing** and human readable.

Disadvantages

- **Verbose**: the tags make files large compared with JSON.
- Parsing and validation need **extra processing time**.

6. Conclusion / Application

XML with schemas is used in **library management systems, SOAP web services, Android layouts and configuration files** (for example `web.xml` in Java web applications).

Unit 2: Cascading Style Sheets (CSS)

Q1. Short Definitions (2 Marks Each)

(a) Three different ways to integrate CSS

1. **Inline CSS:** written in the `style` attribute of an individual HTML element. It has the **highest priority**. `<p style="color: blue; font-size: 16px;">Hello</p>`
2. **Internal (Embedded) CSS:** written inside a `<style>` tag in the `<head>` section. It applies to **one page**.

```
<head>
  <style> p { color: blue; } </style>
</head>
```

3. **External CSS:** written in a separate `.css` file and linked with `<link>`. It applies to **many pages** and is the best for maintenance. `<link rel="stylesheet" href="style.css">`

- **Priority (cascade):** Inline > Internal / External (whichever comes later in the code) > Browser default.

(b) CSS syntax to remove the underline from a hyperlink

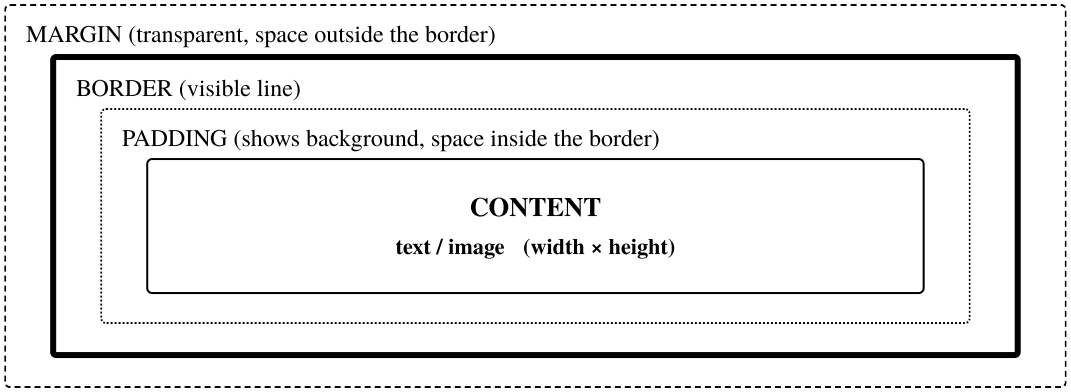
```
a {
    text-decoration: none;
}
/* Optional: show the underline again only on hover */
a:hover {
    text-decoration: underline;
}
```

Q2. Illustrate the CSS Box Model with a proper block diagram, explaining margins, borders, padding and content. (7 Marks)

1. Introduction / Definition

The **CSS Box Model** says that **every HTML element is a rectangular box** made of four layers: **content, padding, border and margin**, from the inside out. It decides the **size, spacing and layout** of elements on a web page.

2. Block Diagram



CSS box model: margin → border → padding → content (outside to inside)

3. Core Explanation / Working

- **Content:**
  - The innermost area, where text, images or child elements are shown.
  - Its size is set by `width` and `height`.
- **Padding:**
  - The space **between the content and the border**.
  - It is **transparent but shows the element's background colour**.
  - Properties: `padding-top`, `padding-right`, `padding-bottom`, `padding-left`, or the shorthand `padding: 10px 20px;`.
- **Border:**
  - The line that wraps around the padding and content.
  - Properties: `border-width`, `border-style` (solid, dashed, dotted), `border-color`, or the shorthand `border: 2px solid black;`.
- **Margin:**
  - The outermost space, **between this element and its neighbours**.
  - It is **fully transparent** (no background).
  - It can be **negative**, and `margin: auto` centres a block horizontally.
  - **Margin collapsing:** the vertical margins of adjacent block elements merge into the larger of the two.

Example and size calculation

```
div {
    width: 300px;
    height: 100px;
    padding: 20px;
    border: 5px solid black;
    margin: 30px;
}
```

- **Total width** = width + left/right padding + left/right border + left/right margin = 300 + 2(20) + 2(5) + 2(30) = **410 px**
- **Total height** = 100 + 2(20) + 2(5) + 2(30) = **210 px**

The `box-sizing` property

- `box-sizing: content-box` (default): `width` covers the content only.
- `box-sizing: border-box`: `width` includes **content + padding + border**, so the box stays at the declared width. This makes responsive layouts easier.

4. Key Characteristics - Padding vs. Margin

Basis	Padding	Margin
Position	Inside the border	Outside the border
Background	Shows element background	Always transparent
Negative values	Not allowed	Allowed
Collapsing	Never collapses	Vertical margins collapse
Click area	Part of the element	Not part of the element
Use	Space inside the box	Space between boxes

5. Advantages & Disadvantages

Advantages

- Gives **precise control** over element size and spacing.
- Applies **uniformly to all elements**, so layouts are predictable.
- `border-box` makes **responsive design** simpler.

Disadvantages

- With the default `content-box`, total size calculations can **confuse beginners** (the declared width is not the real width).
- **Margin collapsing** can cause unexpected gaps.

6. Conclusion / Application

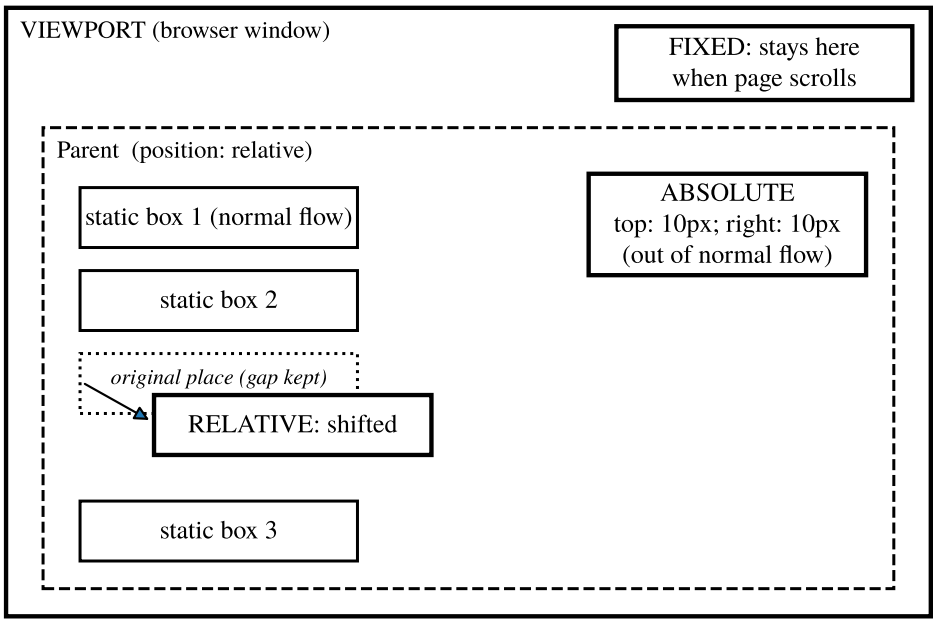
The box model is the **foundation of every CSS layout**, including Flexbox and Grid, and is used to design cards, buttons and page sections on all websites.

Q3. Explain CSS positioning properties (static, relative, absolute, fixed) in detail. (7 Marks)

1. Introduction / Definition

The CSS **`position`** property decides **how an element is placed** in the document. It works together with the **offset properties** `top`, `right`, `bottom` and `left`. The main values are **static, relative, absolute and fixed** (and `sticky` in CSS3).

2. Block Diagram



CSS positioning: static and relative stay in the normal flow; absolute is placed inside the nearest positioned parent; fixed is placed relative to the viewport

3. Core Explanation / Working

(a) `position: static` (default)

- The element follows the **normal document flow** (top to bottom, left to right).
- `top`, `left`, `right`, `bottom` and `z-index` have **no effect**.

```
        p { position: static; }
```

(b) `position: relative`

- The element is positioned **relative to its own normal position**.
- The offset moves the element, but its **original space is kept** (other elements do not move into the gap).
- Often used as a **container (reference point)** for absolutely positioned children.

```
        .box { position: relative; top: 20px; left: 30px; }
        /* moves 20px down and 30px right from where it would normally be */
```

(c) `position: absolute`

- The element is **removed from the normal flow**. Other elements act as if it does not exist.
- It is placed relative to its **nearest positioned ancestor** (an ancestor that is not static). If there is none, it is placed relative to the `<html>` element (initial containing block).
- It **scrolls with the page**.

```
        .parent { position: relative; }
        .badge  { position: absolute; top: 10px; right: 10px; }
```

(d) `position: fixed`

- **Removed from the normal flow** and positioned relative to the **browser viewport**.

- It **does not move** when the page is scrolled.
- Used for sticky headers, “back to top” buttons and chat widgets.

```
        .navbar { position: fixed; top: 0; left: 0; width: 100%; }
```

(e) `position: sticky` (CSS3, extra marks)

- Acts as **relative** until a scroll threshold is reached, then acts as **fixed**. `th { position: sticky; top: 0; }`

(f) `z-index`

- Controls the **stacking order** of positioned elements. A higher value appears in front.

4. Key Characteristics - Comparison

Property	In normal flow?	Positioned relative to	Scrolls with page?	Offsets work?
static	Yes	Normal flow	Yes	No
relative	Yes (space kept)	Its own normal position	Yes	Yes
absolute	**No**	Nearest positioned ancestor	Yes	Yes
fixed	**No**	**Viewport**	**No**	Yes
sticky	Yes	Scroll container (threshold)	Until threshold	Yes

5. Advantages & Disadvantages

Advantages

- Gives **precise placement** of elements like tooltips, badges and modals.
- `fixed` keeps **important navigation always visible**.
- `relative` combined with `absolute` makes building **overlay components** easy.

Disadvantages

- Absolute and fixed elements are **removed from the flow**, so they can **overlap** other content.
- Too much positioning makes layouts **hard to keep responsive**. Flexbox and Grid are better for page layout.

6. Conclusion / Application

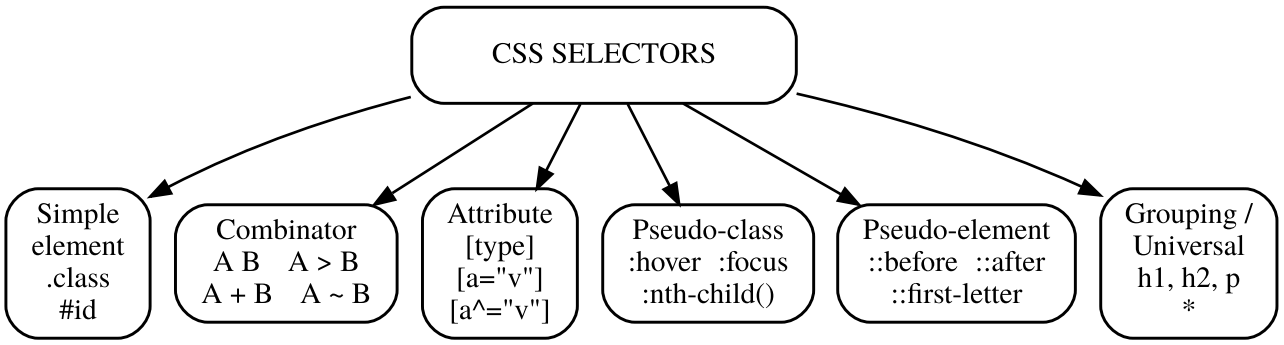
Positioning is used for **fixed navigation bars, dropdown menus, notification badges, modal pop-ups and sticky table headers** on modern websites.

Q4. Discuss different types of CSS selectors and pseudo-classes with examples. (7 Marks)

1. Introduction / Definition

A **CSS selector** is a pattern that **selects the HTML elements** to which a style rule applies. A **pseudo-class** is a keyword added to a selector (with a single colon `:`) that selects elements in a **special state** (like `:hover`) or **position** (like `:first-child`).

2. Block Diagram - Classification of Selectors



Types of CSS selectors

3. Core Explanation / Working

(a) Simple (Basic) Selectors

- **Universal selector (`*`):** selects all elements. `* { margin: 0; padding: 0; }`
- **Element (type) selector:** selects all elements of one type. `p { color: gray; }`
- **Class selector (`.`):** selects elements with a given class. It can be reused on many elements. `.highlight { background: yellow; }`, used as `<p class="highlight">`
- **ID selector (`#`):** selects the **one unique** element with that id. `#header { height: 60px; }`, used as `<div id="header">`
- **Grouping selector (`,`):** applies the same style to several selectors. `h1, h2, h3 { font-family: Arial; }`

(b) Combinator Selectors

- **Descendant (`A B`):** any B inside A. `div p { color: blue; }`
- **Child (`A > B`):** only direct children. `ul > li { list-style: square; }`
- **Adjacent sibling (`A + B`):** the B immediately after A. `h1 + p { font-weight: bold; }`
- **General sibling (`A ~ B`):** every B after A with the same parent. `h1 ~ p { color: green; }`

(c) Attribute Selectors

- `[type]`: elements that have the attribute.
- `input[type="text"]`: the attribute equals a value.
- `a[href^="https"]`: the value **starts with**.
- `a[href$=".pdf"]`: the value **ends with**.
- `img[alt*="logo"]`: the value **contains**.

(d) Pseudo-classes (single colon `:`)

Pseudo-class	Selects	Example
`:link`	Unvisited link	`a:link { color: blue; }`
`:visited`	Visited link	`a:visited { color: purple; }`
`:hover`	Element under the mouse	`a:hover { color: red; }`
`:active`	Element being clicked	`a:active { color: orange; }`
`:focus`	Element with keyboard focus	`input:focus { border: 2px solid blue; }`
`:first-child`	First child of its parent	`li:first-child { font-weight: bold; }`
`:last-child`	Last child	`li:last-child { border: none; }`
`:nth-child(n)`	The nth child	`tr:nth-child(even) { background: #eee; }`
`:not(sel)`	Elements not matching sel	`p:not(.intro) { font-size: 14px; }`
`:checked`	Checked checkbox/radio	`input:checked + label { color: green; }`

- **LVHA order rule:** link styles must be written in the order `:link`, `:visited`, `:hover`, `:active`, or they will override each other.

(e) Pseudo-elements (double colon `::`)

- `p::first-letter { font-size: 200%; }`
- `p::first-line { font-weight: bold; }`
- `h2::before { content: "> "; }` and `h2::after { content: " <"; }`
- `::selection { background: yellow; }`

(f) Specificity (which rule wins)

- **Inline (1000) > ID (100) > Class / attribute / pseudo-class (10) > Element / pseudo-element (1)**. `!important` overrides all of these.

4. Key Characteristics - Class vs. ID Selector, Pseudo-class vs. Pseudo-element

Basis	Class Selector	ID Selector
Symbol	`.`	`#`
Reusability	Many elements	One element per page
Specificity	10	100

Basis	Pseudo-class	Pseudo-element
Syntax	Single colon `:hover`	Double colon `::before`
Targets	A **state** of an existing element	A **part** of an element, or generated content
Example	`:focus`, `:nth-child()`	`::first-letter`, `::after`

5. Advantages & Disadvantages

Advantages

- Selectors give **fine-grained targeting** without adding classes to every element.
- Pseudo-classes add **interactivity** (hover, focus) **without JavaScript**.
- Grouping and reusable classes **reduce code repetition**.

Disadvantages

- Complex selectors and specificity conflicts make CSS **hard to debug**.
- Deep descendant selectors are **slower to match** and fragile when the HTML structure changes.

6. Conclusion / Application

Selectors and pseudo-classes are used in every stylesheet, for example **zebra-striped tables (`:nth-child`), hover effects on navigation menus, and focus highlights on form fields**.

Unit 3: JavaScript, AJAX & Networking

Q1. Short Definitions (2 Marks Each)

(a) What is AJAX and what is its primary advantage for dynamic webpages?

- **AJAX (Asynchronous JavaScript And XML)** is a technique that uses the **`XMLHttpRequest` object (or the `fetch()` API)** to exchange data with the server **in the background**, without reloading the whole web page. The data can be XML, JSON or plain text.
- **Primary advantage: partial page update**. Only the needed part of the page is refreshed, so the page is **faster, uses less bandwidth and gives a smoother user experience**.
- Example: Google search suggestions, the Gmail inbox refresh, and “like” buttons.

(b) Differentiate between Java and JavaScript.

Basis	Java	JavaScript
Type	Object-oriented **programming language**	Object-based **scripting language**
Execution	Compiled to bytecode and run on the **JVM**	**Interpreted / JIT** in the browser (or Node.js)
Typing	**Statically typed** (`int x`)	**Dynamically typed** (`let x`)
Developer	Sun Microsystems (James Gosling)	Netscape (Brendan Eich)
Used for	Desktop, Android, enterprise back end	Web front end, Node.js back end
Concurrency	Multi-threaded	Single-threaded event loop
File extension	`.java` / `.class`	`.js`

(c) Define a Datagram and its characteristics.

- A **datagram** is an **independent, self-contained packet** of data sent over a network **without first setting up a connection**. It carries the full source and destination address. In Java it is used through the `DatagramPacket` and `DatagramSocket` classes (UDP).
- **Characteristics:**
  - **Connectionless:** no handshake is needed before sending.
  - **Unreliable:** delivery, order and duplicate removal are **not guaranteed**.
  - **Fast with low overhead:** no acknowledgements or retransmission.
  - **Fixed maximum size:** each packet is limited (about 64 KB for UDP).

Q2. Explain the JavaScript Document Object Model (DOM). (7 Marks)

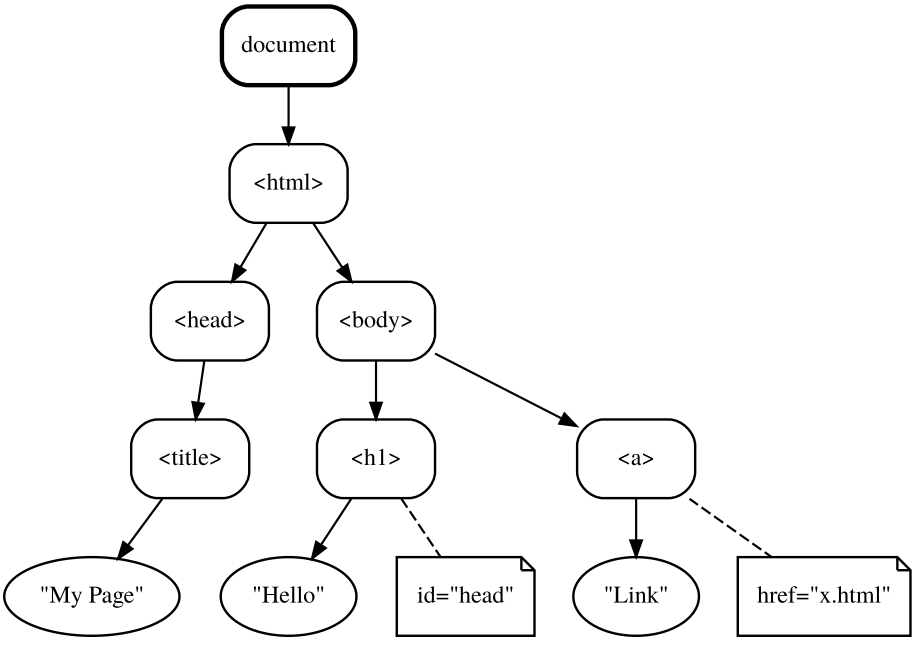
1. Introduction / Definition

The **Document Object Model (DOM)** is a **W3C standard, platform- and language-independent programming interface** that represents an HTML or XML document as a **tree of objects (nodes)**. JavaScript uses the DOM to **access, change, add or delete** elements, attributes, styles and content **dynamically**, without reloading the page.

2. Block Diagram - DOM Tree

HTML source

```
<html>
  <head><title>My Page</title></head>
  <body>
    <h1 id="head">Hello</h1>
    <a href="x.html">Link</a>
  </body>
</html>
```



DOM tree of the page: element, attribute and text nodes

3. Core Explanation / Working

(a) Types of DOM Nodes

- **Document node:** the entire document (`document` object).
- **Element nodes:** HTML tags like `<body>` and `<p>`.
- **Attribute nodes:** attributes like `id` and `href`.
- **Text nodes:** the text inside elements.
- **Comment nodes:** `<!-- -->` comments.

(b) Accessing (Selecting) Elements

```
document.getElementById("head");          // single element by id
document.getElementsByClassName("item");  // HTMLCollection
document.getElementsByTagName("p");       // HTMLCollection
document.querySelector(".item");          // first CSS-selector match
document.querySelectorAll("ul > li");     // NodeList of all matches
```

(c) Modifying Content, Attributes and Styles

```
let h = document.getElementById("head");
h.innerHTML = "<i>Welcome</i>";           // change HTML content
h.textContent = "Welcome";                // change text only
h.setAttribute("class", "title");         // change attribute
h.style.color = "blue";                   // change CSS style
```

(d) Creating and Deleting Nodes

```
let p = document.createElement("p");      // create element
p.textContent = "New paragraph";
document.body.appendChild(p);             // add to the tree
document.body.removeChild(p);             // remove from the tree
```

(e) Traversing the Tree

- `parentNode`, `childNodes`, `children`, `firstChild`, `lastChild`, `nextSibling`, `previousSibling`.

(f) DOM Events (Event Handling)

```
let btn = document.getElementById("btn");
btn.addEventListener("click", function () {
    alert("Button clicked!");
});
```

- Common events: `click`, `submit`, `load`, `change`, `keyup`, `mouseover`.
- **Event propagation:** capturing (top-down), then target, then **bubbling** (bottom-up).

(g) DOM Levels

- **Level 0:** the original browser-specific model.
- **Level 1:** the core and HTML DOM.
- **Level 2:** events, styles, traversal.
- **Level 3:** keyboard events, XPath, load/save.

4. Key Characteristics - `innerHTML` vs. `textContent`, HTML DOM vs. BOM

Basis	`innerHTML`	`textContent`
Reads / writes	HTML markup	Plain text only
Parses tags	Yes	No (tags shown as text)
Security	Risk of **XSS** with user input	Safe
Speed	Slower	Faster

Basis	DOM (Document Object Model)	BOM (Browser Object Model)
Represents	The **page content**	The **browser window**
Root object	`document`	`window`
Standard	W3C standard	No formal standard
Examples	`getElementById`, `createElement`	`alert()`, `location`, `history`, `navigator`

5. Advantages & Disadvantages

Advantages

- Makes web pages **dynamic and interactive** without reloading.
- **Language and platform independent** (usable from JavaScript, Python, Java).
- It is the basis of **AJAX, SPAs and frameworks** (React uses a virtual DOM).

Disadvantages

- Frequent DOM changes cause **reflow and repaint**, which is **slow** for large documents.
- Careless use of `innerHTML` with user input leads to **XSS vulnerabilities**.

6. Conclusion / Application

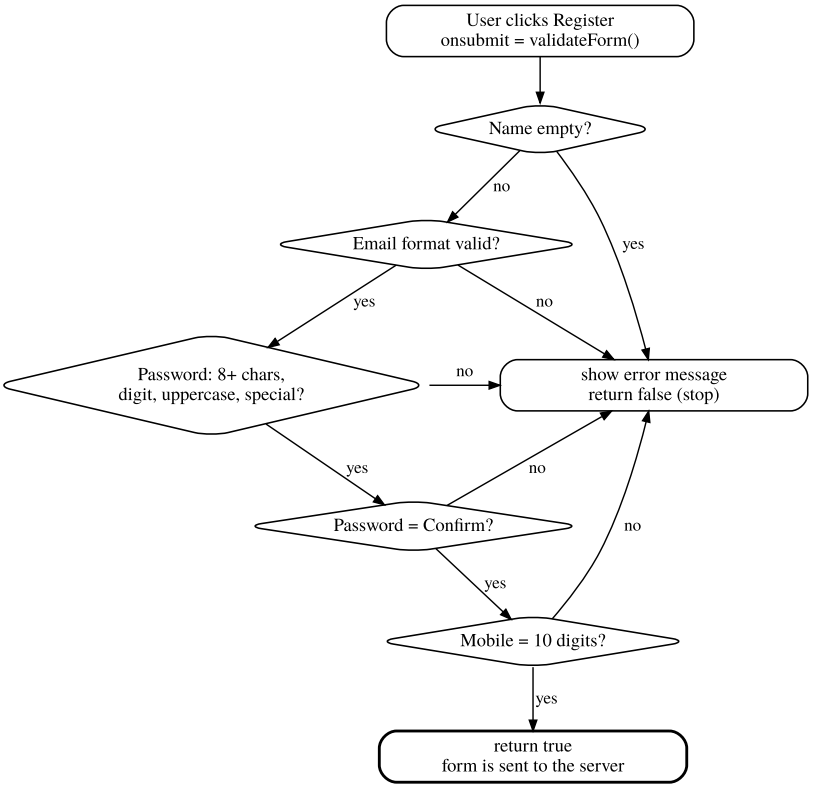
The DOM powers every interactive web feature, such as **live form validation, image sliders, dropdown menus and dynamic content loading** on sites like YouTube and Gmail.

Q3. Write a JavaScript program for client-side form validation (e.g., verifying a password field or empty inputs). (7 Marks)

1. Introduction / Definition

Client-side form validation means checking user input **in the browser, using JavaScript, before the form is sent** to the server. It gives users **instant feedback** and reduces needless server requests. Validation is attached to the form's **`onsubmit`** event, and returning `false` stops the submission.

2. Flowchart - Validation Logic



Client-side validation flow

3. Core Explanation / Working - Program

```
<!DOCTYPE html>
<html>
<head>
  <title>Form Validation</title>
  <style>
    .error { color: red; font-size: 13px; }
  </style>
</head>
<body>
  <h2>Registration</h2>
  <form name="regForm" action="/register" method="post"
        onsubmit="return validateForm()">
    Name: <input type="text" id="name">
    <span class="error" id="nameErr"></span><br><br>

    Email: <input type="text" id="email">
    <span class="error" id="emailErr"></span><br><br>

    Password: <input type="password" id="pwd">
    <span class="error" id="pwdErr"></span><br><br>

    Confirm Password: <input type="password" id="cpwd">
    <span class="error" id="cpwdErr"></span><br><br>

    Mobile: <input type="text" id="mobile">
    <span class="error" id="mobErr"></span><br><br>

    <input type="submit" value="Register">
  </form>

  <script>
    function validateForm() {
      // Read and trim the values
      let name   = document.getElementById("name").value.trim();
      let email  = document.getElementById("email").value.trim();
      let pwd    = document.getElementById("pwd").value;
      let cpwd   = document.getElementById("cpwd").value;
      let mobile = document.getElementById("mobile").value.trim();
      let valid  = true;

      // Clear old error messages
      document.querySelectorAll(".error")
              .forEach(function (e) { e.textContent = ""; });

      // 1. Empty-field check
      if (name === "") {
        document.getElementById("nameErr").textContent = "Name is required";
        valid = false;
      }

      // 2. Email format check
      let emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
      if (!emailPattern.test(email)) {
        document.getElementById("emailErr").textContent = "Enter a valid email";
        valid = false;
      }

      // 3. Password strength check: min 8 chars, 1 uppercase,
      //    1 digit, 1 special character
      let pwdPattern = /^(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#$%^&*]).{8,}$/;
      if (!pwdPattern.test(pwd)) {
        document.getElementById("pwdErr").textContent =
          "Min 8 chars with 1 uppercase, 1 digit and 1 special char";
        valid = false;
      }
```

```
            // 4. Confirm password check
            if (pwd !== cpwd) {
              document.getElementById("cpwdErr").textContent = "Passwords do not match";
              valid = false;
            }

            // 5. Mobile number check (exactly 10 digits)
            if (!/^[0-9]{10}$/.test(mobile)) {
              document.getElementById("mobErr").textContent = "Enter a 10-digit number";
              valid = false;
            }

            return valid;   // false stops the form submission
          }
        </script>
      </body>
      </html>
```

Working

- `onsubmit="return validateForm()"` runs the function **before** the form is sent.
- `.value.trim()` reads the input and removes extra spaces.
- `RegExp.test()` checks the format (email, password strength, mobile).
- Error messages appear in `<span>` elements through `textContent`, next to the wrong field.
- If any check fails, the function returns **`false`** and the submission is **cancelled**.

4. Key Characteristics - Client-side vs. Server-side Validation

Basis	Client-side Validation	Server-side Validation
Runs on	Browser (JavaScript)	Web server (Servlet, Node.js)
Speed	**Instant** feedback	Needs a round trip
Security	**Can be bypassed** (JS can be disabled)	**Secure** and mandatory
Server load	Reduced	Increased
Purpose	User convenience	Data integrity and security

5. Advantages & Disadvantages

Advantages

- **Immediate feedback** improves the user experience.
- **Reduces server load** and network traffic by catching errors early.
- Regular expressions allow **flexible format checks**.

Disadvantages

- It can be **bypassed** by disabling JavaScript or editing requests, so server checks are still needed.
- Validation logic is **visible** in the page source.

6. Conclusion / Application

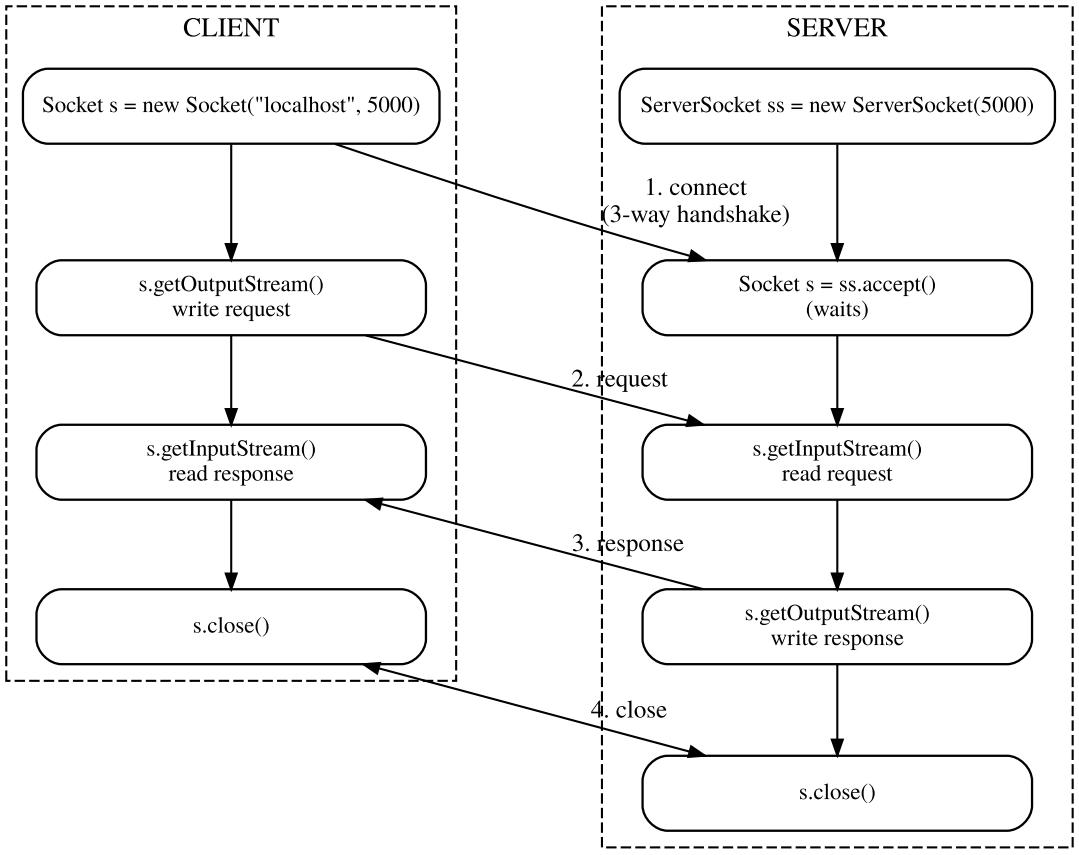
Client-side validation is used on every **login, sign-up, payment and contact form**, for example checking the password strength on a Gmail or bank account registration page.

Q4. Explain TCP/IP client sockets and write the constructor and methods used to create a client socket. (7 Marks)

1. Introduction / Definition

A **socket** is one **endpoint of a two-way communication link** between two programs on a network, identified by an **IP address and a port number**. A **TCP/IP client socket** (Java class **`java.net.Socket`**) connects to a server socket and gives a **reliable, connection-oriented, stream-based** link using TCP.

2. Block Diagram - TCP Client-Server Socket Communication



TCP client-server socket communication

3. Core Explanation / Working

(a) Steps to Create a TCP Client

1. Create a `Socket` object with the server's **host name or IP** and **port**. This starts the TCP connection (three-way handshake).
2. Get the **input and output streams** from the socket.
3. **Send** a request through the output stream and **receive** the response through the input stream.
4. **Close** the streams and the socket.

(b) Constructors of the `Socket` Class

Constructor	Description
`Socket(String host, int port)`	Connects to the given host name and port. Throws `UnknownHostException` or `IOException`.
`Socket(InetAddress address, int port)`	Connects using an `InetAddress` object and port.
`Socket(String host, int port, InetAddress localAddr, int localPort)`	Connects and also binds to a given **local** address and port.
`Socket(InetAddress addr, int port, InetAddress localAddr, int localPort)`	Same, using an InetAddress for the server.
`Socket()`	Creates an **unconnected** socket. Call `connect()` later.

(c) Important Methods of the `Socket` Class

Method	Purpose
`InputStream getInputStream()`	Returns the stream for **reading** data from the server
`OutputStream getOutputStream()`	Returns the stream for **writing** data to the server
`InetAddress getInetAddress()`	IP address of the **remote** host
`int getPort()`	Remote port number
`int getLocalPort()`	Local port the socket is bound to
`InetAddress getLocalAddress()`	Local IP address
`void connect(SocketAddress ep, int timeout)`	Connects an unconnected socket, with a timeout
`void setSoTimeout(int ms)`	Sets the read timeout
`boolean isConnected()` / `isClosed()`	Checks the connection state
`void close()`	Closes the socket and releases resources

(d) Program - TCP Client in Java

```
import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        try {
            // 1. Create the client socket (connects to the server)
            Socket s = new Socket("localhost", 5000);
            System.out.println("Connected to " + s.getInetAddress()
                               + " on port " + s.getPort());

            // 2. Get the I/O streams
            PrintWriter out = new PrintWriter(s.getOutputStream(), true);
            BufferedReader in = new BufferedReader(
                                new InputStreamReader(s.getInputStream()));

            // 3. Send a request and read the response
            out.println("Hello Server");
            String reply = in.readLine();
            System.out.println("Server says: " + reply);

            // 4. Close the resources
            in.close();
            out.close();
            s.close();
        } catch (UnknownHostException e) {
            System.out.println("Host not found: " + e.getMessage());
        } catch (IOException e) {
            System.out.println("I/O error: " + e.getMessage());
        }
    }
}
```

Matching server, in short: `ServerSocket ss = new ServerSocket(5000); Socket s = ss.accept();`, then read with `s.getInputStream()` and reply with `s.getOutputStream()`.

4. Key Characteristics - TCP Socket vs. UDP Datagram Socket

Basis	TCP (`Socket`, `ServerSocket`)	UDP (`DatagramSocket`, `DatagramPacket`)
Connection	**Connection-oriented**	**Connectionless**
Reliability	Guaranteed delivery and order	Not guaranteed
Data unit	Continuous byte **stream**	Independent **packets**
Speed	Slower (handshake, ACKs)	Faster
Header size	20 bytes	8 bytes
Use case	HTTP, FTP, email, chat	Video streaming, DNS, online games

5. Advantages & Disadvantages

Advantages

- **Reliable, ordered** delivery with error checking (TCP).
- The **stream-based API** makes reading and writing as easy as file I/O.
- **Platform independent** in Java.

Disadvantages

- **Connection set-up overhead** (three-way handshake) makes it slower than UDP.
- `accept()` and `read()` **block**, so threads are needed for many clients.

6. Conclusion / Application

TCP client sockets are the basis of **web browsers, chat applications, FTP clients and database drivers (JDBC)**, which all open a socket to talk to their servers.

Unit 4: Java Beans & Enterprise Java Beans (EJB)

Q1. Short Definitions (2 Marks Each)

(a) What exactly is a Java Bean?

- A **Java Bean** is a **reusable software component** written in Java that can be **visually manipulated in builder tools** (such as NetBeans). It follows the JavaBeans API conventions.
- **Rules (conventions) a class must follow:**
  1. It must have a **public no-argument constructor**.
  2. Its properties must be **private**, accessed through public **getter/setter** methods (`getName()`, `setName()`, and `isActive()` for booleans).
  3. It should implement **`java.io.Serializable`**, so its state can be saved and restored.

```
public class Student implements java.io.Serializable {
    private String name;
    public Student() { }                          // no-arg constructor
    public String getName() { return name; }      // getter
    public void setName(String n) { name = n; }   // setter
}
```

(b) What is the role of introspection in a Java Bean?

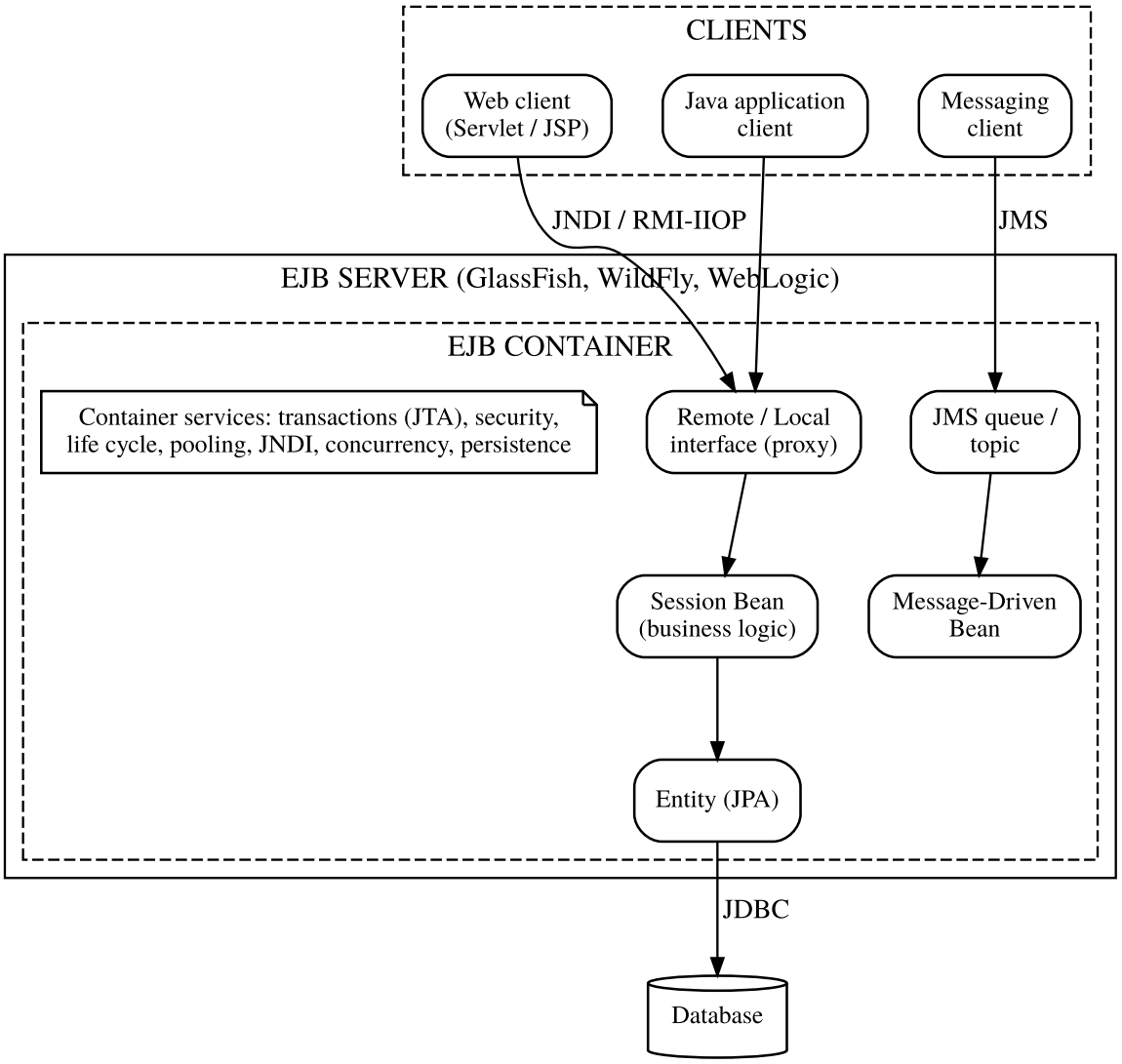
- **Introspection** is the process by which a builder tool **automatically discovers a bean's properties, methods and events at run time**, without seeing its source code.
- It is done with the **`java.beans.Introspector`** class, which uses **Java Reflection** and the **naming conventions** (get/set/is), or an explicit **`BeanInfo`** class supplied by the developer.
- Role: it lets IDEs **show and edit bean properties visually** and **connect events**, which makes beans truly reusable "plug-and-play" components. `BeanInfo info = Introspector.getBeanInfo(Student.class);`

Q2. Explain the complete EJB Architecture. (7 Marks)

1. Introduction / Definition

Enterprise JavaBeans (EJB) is a **server-side, component-based architecture** (part of Jakarta EE / Java EE) for building **distributed, transactional, secure and scalable** business applications. Business logic is written in **enterprise beans**, and these run inside an **EJB container**. The container automatically provides system-level services like transactions, security and persistence.

2. Block Diagram - EJB Architecture



EJB architecture

3. Core Explanation / Working

(a) Components of the EJB Architecture

- **EJB Server:** the application server that provides the **run-time environment** and hosts one or more containers. Examples: GlassFish, JBoss/WildFly, WebLogic.
- **EJB Container:** a layer inside the server that **manages enterprise beans**. The client never talks to the bean directly. Every call goes **through the container**, which adds its services.
- **Enterprise Beans:** the components that hold the **business logic**.
  - **Session Beans:** do tasks for a client (stateless, stateful, singleton).
  - **Message-Driven Beans (MDB):** process **asynchronous JMS messages**.
  - **Entity Beans:** represent persistent data. In EJB 3.x they are replaced by **JPA entities**.
- **Interfaces:**
  - **Remote interface (`@Remote`):** for clients in **another JVM**, using RMI-IIOP.
  - **Local interface (`@Local`):** for clients in the **same JVM**, which is faster.
  - **Home interface** (EJB 2.x): a factory for creating and finding beans. It is no longer needed in EJB 3.x.
- **Deployment Descriptor (`ejb-jar.xml`) or annotations:** configure transactions, security and resources.
- **JNDI (Java Naming and Directory Interface):** clients **look up** beans by name.

(b) Services Provided by the Container

- **Transaction management (JTA):** automatic commit/rollback (`@TransactionAttribute`).
- **Security:** role-based access control (`@RolesAllowed`).
- **Life-cycle management and instance pooling:** creates, pools and destroys beans.
- **Concurrency control:** thread safety.
- **Persistence:** through JPA.
- **Remote access and location transparency.**

(c) Working (request flow)

1. The client **looks up** the bean through **JNDI**, or gets it through `@EJB` injection.
2. The container returns a **proxy (stub)** that implements the business interface.
3. The client calls a business method on the proxy.
4. The container **intercepts** the call, starts a transaction, checks security and gets a bean instance from the **pool**.
5. The bean runs the logic (and may use the database through JPA/JDBC).
6. The container **commits** the transaction and returns the result to the client.

(d) Example (EJB 3.x)

```
@Stateless
public class BankBean implements BankRemote {
    public double getBalance(int accNo) { /* business logic */ return 5000.0; }
}

@Remote
public interface BankRemote { double getBalance(int accNo); }
```

4. Key Characteristics - Types of Enterprise Beans

Basis	Session Bean	Message-Driven Bean	Entity (JPA)
Purpose	Business logic / tasks	Asynchronous message processing	Persistent data
Invocation	Synchronous (method call)	Asynchronous (JMS message)	Through EntityManager
Client state	Stateful or stateless	Stateless	Represents a DB row
Annotation	`@Stateless`, `@Stateful`, `@Singleton`	`@MessageDriven`	`@Entity`
Survives server crash	No	No	**Yes** (stored in DB)

5. Advantages & Disadvantages

Advantages

- The developer writes **only business logic**. The container handles transactions, security and pooling.
- **Scalable and distributed**: beans can be accessed remotely and clustered.
- **Portable** across any Java EE-compliant application server.

Disadvantages

- Needs a **heavyweight application server**, which uses more resources.

- **Complex** compared with lightweight frameworks like Spring Boot, with a steeper learning curve.
- Remote calls add **network overhead**.

6. Conclusion / Application

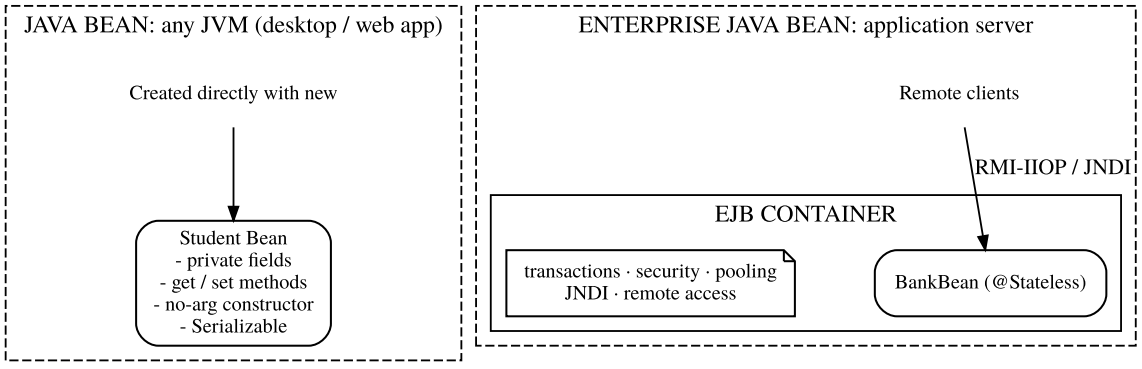
EJB is used in **banking, insurance, airline reservation and large enterprise ERP systems**, which need distributed transactions and high security.

Q3. Differentiate between a Java Bean and an Enterprise Java Bean (EJB). (7 Marks)

1. Introduction / Definition

A **Java Bean** is a simple, **client-side or general-purpose reusable component** that follows naming conventions and is used mainly in builder tools and for data transfer. An **Enterprise Java Bean (EJB)** is a **server-side, distributed business component** that runs inside an **EJB container**, which provides transaction, security and life-cycle services.

2. Block Diagram - Where Each Runs



Where a Java Bean and an EJB run

3. Core Explanation / Working

Java Bean

- A **plain Java class** that follows conventions: a no-arg constructor, private properties with getters/setters, and `Serializable`.
- Created directly with `new`, or with `<jsp:useBean>` in JSP.
- Used for **GUI components (Swing/AWT)**, **data holders (DTOs)** and **JSP form beans**.
- Supports **properties, events, introspection, customization and persistence**.

Enterprise Java Bean

- A **managed component**. It is **never created with `new`** by the client. The **container** creates, pools and destroys it.
- It is accessed through **business interfaces** (local or remote) using **JNDI lookup** or `@EJB` injection.
- Types: **Session beans** (stateless, stateful, singleton) and **Message-Driven beans**.
- Gets **declarative services**: transactions, security, concurrency and remoting.

4. Key Characteristics - Comparison Table

Basis	Java Bean	Enterprise Java Bean (EJB)
Nature	Simple reusable component	Distributed **server-side** business component
Runs in	Any JVM (no container needed)	Only inside an **EJB container** (application server)
Created by	Programmer (`new`) or builder tool	**Container**
API / package	`java.beans`	`javax.ejb` / `jakarta.ejb`
Interfaces needed	None	Business interface (`@Local` / `@Remote`)
Remote access	Not supported	Supported (RMI-IIOP)
Transactions and security	Must be coded manually	**Provided by the container** (declarative)
Persistence	Through serialization	Through JPA / database
Visual (GUI) use	Yes (builder tools)	No visual representation
Examples	Swing button, Student DTO	Banking service bean, order-processing bean
Complexity	Low	High

5. Advantages & Disadvantages

Java Bean

- *Advantages:* **simple and lightweight**; reusable in any Java application; easy to use in JSP and IDEs.
- *Disadvantages:* **no built-in enterprise services** (transactions, security); cannot be called remotely.

EJB

- *Advantages:* **container-managed services**; supports **distributed and scalable** applications.
- *Disadvantages:* needs a **heavy application server**; **more complex** to develop and deploy.

6. Conclusion / Application

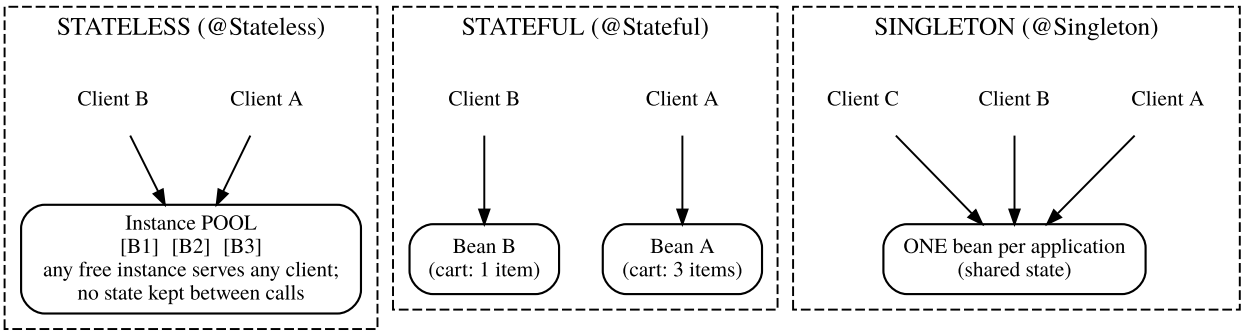
Java Beans are used as **data and UI components in desktop and JSP applications**, while EJBs implement **core business services in enterprise banking and e-commerce back ends**.

Q4. Explain session beans and detail their various types (Stateful vs. Stateless). (7 Marks)

1. Introduction / Definition

A **session bean** is an enterprise bean that **performs business tasks for a client** during a session, such as calculations, bank transfers or order processing. It is **not persistent**: it does not represent data in the database and it is lost when the server shuts down. There are three types: **Stateless, Stateful and Singleton**.

2. Block Diagram - Session Bean Types



Session bean types

Life cycle of a Stateful Session Bean



Life cycle of a stateful session bean (a stateless bean has only Does Not Exist and Ready)

3. Core Explanation / Working

(a) Stateless Session Bean (`@Stateless`)

- Keeps **no conversational state** for the client between method calls.
- Any free instance from the **pool** can serve any client, so instances are **shared**.
- **Highly scalable**, because a few instances serve thousands of clients.
- Life cycle: Does Not Exist, then Method-Ready Pool. Callbacks: `@PostConstruct`, `@PreDestroy`.
- Example: currency converter, tax calculator, sending email.

```
@Stateless
public class CalculatorBean {
    public int add(int a, int b) { return a + b; }
}
```

(b) Stateful Session Bean (`@Stateful`)

- Keeps **conversational state** for **one specific client** across many method calls.
- There is **one instance per client**, and the instances are not shared.
- If the bean is idle, the container can **passivate** it (save its state to disk) and **activate** it later.
- It ends when the client calls a method marked `@Remove`, or after a timeout.
- Callbacks: `@PostConstruct`, `@PrePassivate`, `@PostActivate`, `@PreDestroy`.
- Example: **shopping cart**, online exam session, multi-step booking.

```
@Stateful
public class CartBean {
    private List<String> items = new ArrayList<>();
    public void addItem(String item) { items.add(item); }
    public List<String> getItems() { return items; }
    @Remove
    public void checkout() { /* place order, bean is removed */ }
}
```

(c) Singleton Session Bean (`@Singleton`) (EJB 3.1)

- **Only one instance per application**, shared by all clients.
- Used for **shared application state**: cache, configuration, counters.
- Concurrency is controlled with `@Lock(READ)` and `@Lock(WRITE)`. With `@Startup` it is created when the application starts.

4. Key Characteristics - Stateful vs. Stateless

Basis	Stateless Session Bean	Stateful Session Bean
Conversational state	**Not kept** between calls	**Kept** for one client
Instance-to-client	Shared from a pool	**One instance per client**
Scalability	**Very high**	Lower (more memory)
Passivation / activation	Not needed	Supported
Life-cycle states	Does Not Exist, Ready	Does Not Exist, Ready, **Passive**
Annotation	`@Stateless`	`@Stateful` (+ `@Remove`)
Performance	Faster	Slower
Example	Tax calculator, email sender	Shopping cart, booking wizard

5. Advantages & Disadvantages

Advantages

- Business logic is **separated** from the presentation layer.
- The container provides **transactions, security and pooling** automatically.
- **Stateless beans scale** very well. **Stateful beans make multi-step workflows simple**.

Disadvantages

- Stateful beans use **more memory** and passivation adds **I/O overhead**.
- Session beans are **not persistent**, so their data is lost on a crash unless it is saved to a database.

6. Conclusion / Application

Stateless beans implement **payment and calculation services**, while stateful beans implement **shopping carts on e-commerce sites and step-by-step ticket booking** in reservation systems.

Unit 5: Modern Backend (Servlets, JSP, Node.js & MongoDB)

Q1. Short Definitions (2 Marks Each)

(a) What is Node.js and what is its primary use case?

- **Node.js** is an **open-source, cross-platform JavaScript run-time environment** built on Google Chrome's **V8 engine**. It lets JavaScript run **on the server**, outside the browser. It uses a **single-threaded, event-driven, non-blocking I/O** model.

- **Primary use case:** building **fast, scalable network applications that handle many requests at once and spend most of their time on I/O**, such as **REST APIs, real-time chat apps (Socket.io), streaming services and microservices**.

(b) What is the purpose of the `HttpServletRequest` interface?

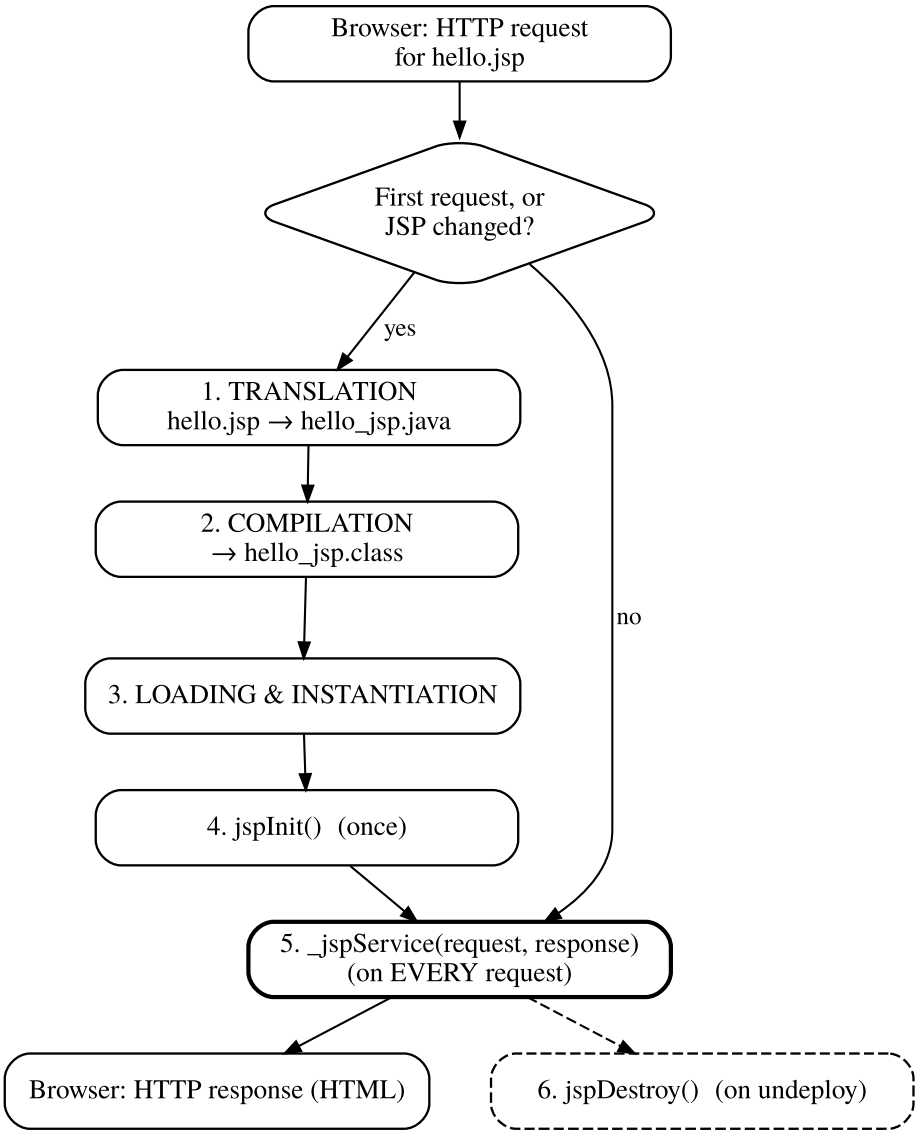
- `javax.servlet.http.HttpServletRequest` (it extends `ServletRequest`) **represents the client's HTTP request**. The servlet container creates it and passes it to the servlet's `service()` / `doGet()` / `doPost()` methods.
- **Purpose:** it lets the servlet **read everything the client sent**:
  - Form parameters: `getParameter("name")`, `getParameterValues()`
  - Headers: `getHeader("User-Agent")`
  - Cookies and session: `getCookies()`, `getSession()`
  - Request information: `getMethod()`, `getRequestURI()`, `getRemoteAddr()`
  - Request attributes for forwarding: `setAttribute()`, `getAttribute()`

Q2. Discuss the compilation process of JSP files into Servlets and how they are executed by the container. (7 Marks)

1. Introduction / Definition

JSP (JavaServer Pages) is a server-side technology for creating dynamic web pages by **embedding Java code in HTML**. A JSP page is **not run directly**. The **JSP container (for example Tomcat's Jasper engine)** first **translates it into a Java Servlet**, **compiles** that servlet into bytecode, and then **runs** it like any other servlet.

2. Block Diagram - JSP Processing / Life Cycle



JSP processing: translation and compilation happen only on the first request (or after the JSP changes)

3. Core Explanation / Working

Phase 1 - Translation

- On the **first request** (or after the JSP file changes), the JSP engine **parses** `hello.jsp` and creates the Java source file `hello_jsp.java`.
- The generated class extends `HttpJspBase`, which implements the `HttpJspPage` interface.
- Mapping of JSP elements to servlet code:
  - Static HTML becomes `out.write("...")` statements.
  - Scriptlet `<% code %>` is copied into `_jspService()`.
  - Expression `<%= expr %>` becomes `out.print(expr)`.
  - Declaration `<%! int x; %>` becomes a class-level field or method.
  - Directive `<%@ page import="java.util.*" %>` becomes `import` statements and settings.

Phase 2 - Compilation

- The container compiles `hello_jsp.java` with the Java compiler into **`hello_jsp.class`** (bytecode).
- Syntax errors in the JSP appear at this stage as compilation errors.

Phase 3 - Class Loading and Instantiation

- The class loader **loads** `hello_jsp.class`, and the container creates **one instance** of it.

Phase 4 - Initialization: `jspInit()`

- Called **once**, to set up resources like database connections. It can be overridden in a declaration.

Phase 5 - Request Processing: `_jspService(request, response)`

- Called for **every request**, each in a **separate thread**. It produces the dynamic HTML response.
- **Implicit objects** are available here: `request`, `response`, `out`, `session`, `application`, `config`, `pageContext`, `page`, `exception`.
- It **cannot be overridden** (the underscore marks it as container-generated).

Phase 6 - Destruction: `jspDestroy()`

- Called **once** when the JSP is removed or the server shuts down, to release resources.

Example

```
<%@ page contentType="text/html" %>
<html><body>
  <% String user = request.getParameter("name"); %>
  <h2>Welcome, <%= user %>!</h2>
</body></html>
```

The generated servlet (simplified):

```
public final class hello_jsp extends HttpJspBase {
  public void _jspService(HttpServletRequest request, HttpServletResponse response)
        throws IOException, ServletException {
    JspWriter out = pageContext.getOut();
    out.write("<html><body>\n");
    String user = request.getParameter("name");
    out.write("<h2>Welcome, ");
    out.print(user);
    out.write("!</h2>\n</body></html>");
  }
}
```

- **Later requests skip translation and compilation.** The already-loaded instance runs `_jspService()` directly, so only the first request is slow.

4. Key Characteristics - JSP vs. Servlet

Basis	JSP	Servlet
Code style	Java inside HTML	HTML inside Java (`out.println`)
Best for	**Presentation (View)**	**Business logic (Controller)**
Translation step	Yes (JSP to Servlet)	No
First request	Slower (translate + compile)	Faster
Recompile on change	**Automatic**	Manual recompile and redeploy
Implicit objects	Available (`request`, `out`, `session`)	Must be obtained manually
Life-cycle methods	`jspInit`, `_jspService`, `jspDestroy`	`init`, `service`, `destroy`

5. Advantages & Disadvantages

Advantages

- **Easier to write and maintain** than servlets for HTML-heavy pages.
- **Automatic recompilation** when the JSP changes.
- Supports **tag libraries (JSTL)** and Expression Language, which reduce Java code in pages.

Disadvantages

- The **first request is slow**, because of translation and compilation.
- Mixing Java and HTML makes pages **hard to debug and maintain** if scriptlets are overused.

6. Conclusion / Application

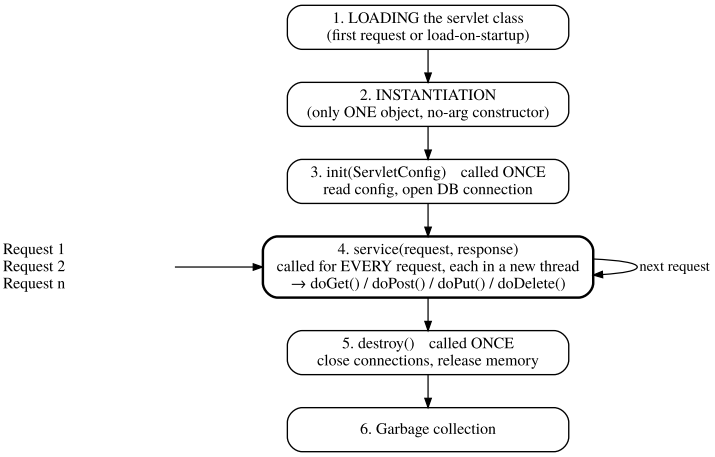
JSP is used as the **View layer in MVC web applications** (with servlets as controllers), for example in **online banking portals and college result systems** running on Apache Tomcat.

Q3. Explain the Servlet lifecycle methods (`init`, `service`, `destroy`). (7 Marks)

1. Introduction / Definition

A **Servlet** is a server-side Java class that **handles HTTP requests and generates dynamic responses** inside a **servlet container** (for example Apache Tomcat). The **servlet life cycle** is the sequence of stages from the servlet's **creation to its destruction**. The container controls it through the three methods of the `javax.servlet.Servlet` interface: **`init()`, `service()` and `destroy()`**.

2. Block Diagram - Servlet Life Cycle



Servlet life cycle

3. Core Explanation / Working

Step 1 & 2 - Loading and Instantiation

- The container **loads** the servlet class when the **first request** arrives, or at **server start-up** if `<load-on-startup>` is set in `web.xml`.
- It creates **exactly one instance** using the public no-arg constructor.

Step 3 - `init()` Method

- Signature: `public void init(ServletConfig config) throws ServletException`
- Called **only once**, right after instantiation and **before any request** is handled.
- Used for **one-time set-up**: reading init parameters, opening database connections, loading configuration.
- If it throws `ServletException`, the servlet is not put into service.

Step 4 - `service()` Method

- Signature: `public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException`
- Called **for every client request**. The container creates a **new thread** for each request, all on the **same instance**, so shared fields must be thread-safe.
- In `HttpServlet`, `service()` checks the HTTP method and **dispatches** to `doGet()`, `doPost()`, `doPut()`, `doDelete()` and so on.
- Developers usually override `doGet()` and `doPost()`, not `service()`.

Step 5 - `destroy()` Method

- Signature: `public void destroy()`
- Called **only once**, when the servlet is taken out of service (server shutdown, application undeploy or reload).
- Used to **release resources**: close connections, stop threads, save state.
- After this, the instance is **garbage-collected**.

Example Program

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LifeCycleServlet extends HttpServlet {

    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        System.out.println("init(): servlet initialized (called once)");
    }

    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        String name = req.getParameter("name");
        out.println("<h2>Hello, " + name + "</h2>");   // runs per request
    }

    public void destroy() {
        System.out.println("destroy(): servlet destroyed (called once)");
    }
}
```

Deployment descriptor (`web.xml`)

```
<servlet>
  <servlet-name>life</servlet-name>
  <servlet-class>LifeCycleServlet</servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>life</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

(Alternatively, use the annotation `@WebServlet("/hello")` in Servlet 3.0+.)

4. Key Characteristics - Life-Cycle Methods

Basis	`init()`	`service()`	`destroy()`
Called by	Container	Container	Container
How many times	**Once**	**Once per request**	**Once**
When	After instantiation	On each client request	Before removal
Parameters	`ServletConfig`	`ServletRequest`, `ServletResponse`	None
Purpose	Initialize resources	Process the request, build the response	Release resources
Threading	Single thread	**Multi-threaded**	Single thread

5. Advantages & Disadvantages (of the servlet life-cycle model)

Advantages

- **One instance, many threads**: much faster and lighter than CGI, which starts a new process per request.
- **One-time initialization** in `init()` avoids repeated costly set-up.

- **Clean resource release** in `destroy()` prevents memory leaks.

Disadvantages

- Instance variables are shared by all threads, so **thread-safety problems** (race conditions) must be handled.
- The **first request is slower** (loading and `init()`) unless `load-on-startup` is used.

6. Conclusion / Application

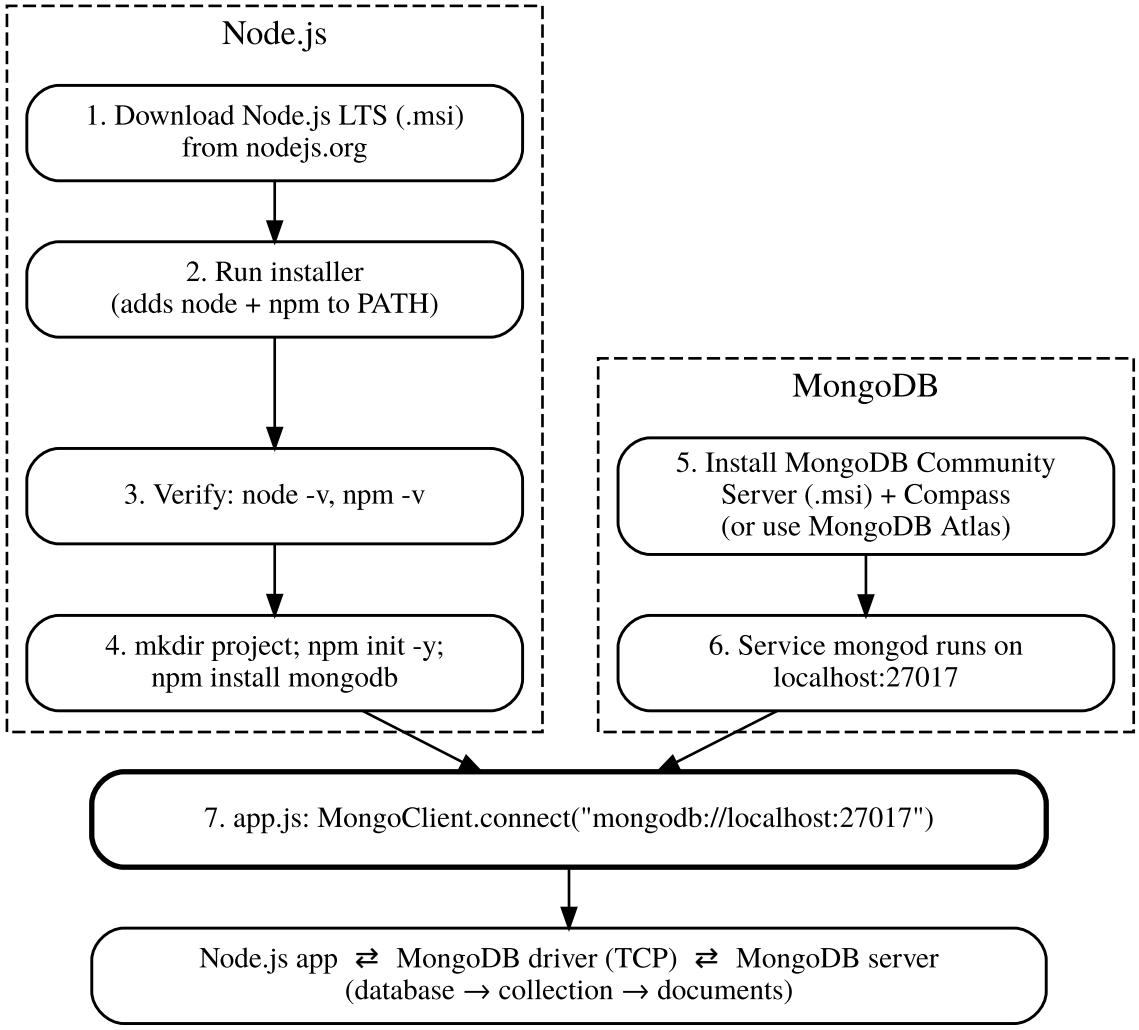
The servlet life cycle is the basis of **every Java web framework (Spring MVC's DispatcherServlet, Struts)** and of login handlers, form processors and REST endpoints on Apache Tomcat.

Q4. Outline the step-by-step process to set up Node.js in Windows and create a MongoDB database connection. (7 Marks)

1. Introduction / Definition

Node.js is a JavaScript run time for building server-side applications, and **MongoDB** is a **NoSQL, document-oriented database** that stores data as **JSON-like BSON documents** in **collections**. Node.js connects to MongoDB using the official **`mongodb` driver** or the **Mongoose ODM** library.

2. Block Diagram - Set-up and Connection Flow



Setting up Node.js and MongoDB on Windows

3. Core Explanation / Working

Part A - Installing Node.js on Windows

1. Open **https://nodejs.org** and download the **LTS (Long Term Support)** Windows Installer (`.msi`, 64-bit).
2. Run the installer and accept the licence. Keep the default path `C:\Program Files\nodejs\`. Make sure **"Add to PATH"** and the **npm package manager** are selected. Click **Install**.
3. Open **Command Prompt** and check the installation:

```
node -v      # e.g. v20.x.x
npm -v       # e.g. 10.x.x
```

4. Create a project folder and initialise it:

```
mkdir myapp
cd myapp
npm init -y              # creates package.json
npm install mongodb      # official MongoDB driver
```

Part B - Installing and Starting MongoDB

5. Download **MongoDB Community Server** (`.msi`) from **mongodb.com**. Choose **"Complete"** set-up and **"Install MongoDB as a Service"**. Optionally install **MongoDB Compass** (GUI).
6. The MongoDB service starts automatically and listens on **`localhost:27017`**. Check it with `mongosh` in the terminal.
    - *Alternative:* create a free cloud cluster on **MongoDB Atlas** and copy its connection string (`mongodb+srv://...`).

Part C - Creating the Database Connection (`app.js`)

```
// app.js
const { MongoClient } = require("mongodb");

const url = "mongodb://127.0.0.1:27017";   // connection string
const client = new MongoClient(url);
const dbName = "collegeDB";

async function main() {
    try {
        // 1. Connect to the MongoDB server
        await client.connect();
        console.log("Connected successfully to MongoDB");

        // 2. Select the database (created automatically on first insert)
        const db = client.db(dbName);
        const students = db.collection("students");

        // 3. Insert a document
        const result = await students.insertOne(
            { roll: 101, name: "Harsh", branch: "CSE", year: 3 });
        console.log("Inserted id:", result.insertedId);

        // 4. Read documents
        const data = await students.find({ branch: "CSE" }).toArray();
        console.log(data);
    } catch (err) {
        console.error("Connection error:", err);
    } finally {
        // 5. Close the connection
        await client.close();
    }
}

main();
```

7. Run the program:

```
node app.js
```

Output:

```
Connected successfully to MongoDB
Inserted id: 6650f1c2a4...
[ { _id: ..., roll: 101, name: 'Harsh', branch: 'CSE', year: 3 } ]
```

Alternative using Mongoose (schema-based ODM)

```
const mongoose = require("mongoose");          // npm install mongoose
mongoose.connect("mongodb://127.0.0.1:27017/collegeDB")
        .then(() => console.log("Mongoose connected"))
        .catch(err => console.log(err));
const Student = mongoose.model("Student",
        new mongoose.Schema({ roll: Number, name: String }));
```

4. Key Characteristics - MongoDB (NoSQL) vs. MySQL (SQL)

Basis	MongoDB	MySQL
Data model	**Documents** (JSON/BSON)	**Tables** (rows and columns)
Schema	**Flexible / schema-less**	Fixed schema
Query language	JavaScript-like methods (`find`, `insertOne`)	SQL
Scaling	**Horizontal** (sharding)	Mainly vertical
Joins	Limited (`$lookup`)	Full support
Best with	Node.js (MERN / MEAN stack)	PHP, Java (LAMP)

5. Advantages & Disadvantages

Advantages

- **JavaScript end to end**: Node.js and MongoDB both use JSON, so no data conversion is needed.
- **Non-blocking async I/O** handles many concurrent database requests efficiently.
- **Flexible schema** speeds up development, and data scales horizontally.

Disadvantages

- No full **ACID joins** like SQL. Complex relational queries are harder.
- Node.js is **not suited to CPU-heavy tasks**, which block its single thread.
- The flexible schema can lead to **inconsistent data** without validation (Mongoose helps).

6. Conclusion / Application

Node.js with MongoDB forms the back end of the **MERN/MEAN stack**, used in **real-time chat apps, e-commerce sites and social media platforms** (for example parts of Netflix, Uber and LinkedIn).
